

Series: Hardening Apache HTTPD in Docker (Part 1 of 4)

Author: H. Brister Wooley

Published: 2026-01-05

Series: Hardening Apache HTTPD in Docker (1/4)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-01-05-hardening-apache-docker-part1.md>

Grab a coffee. No, seriously – this is the conversation you wish you'd had three years ago before your first container escape incident.

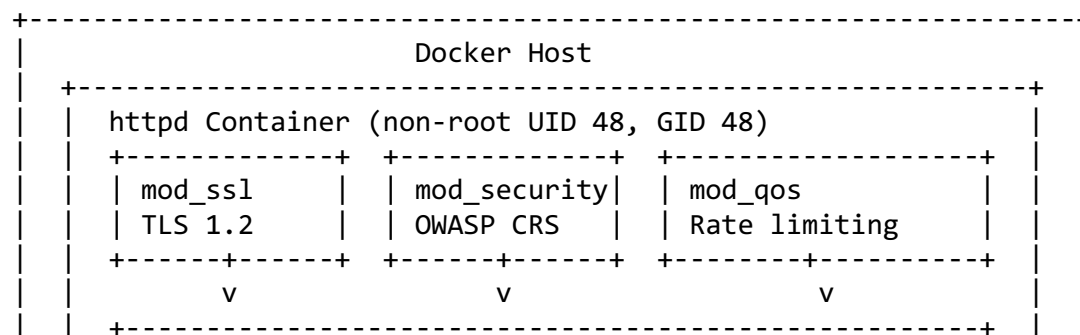
You've containerized Apache. Image builds, pod runs, health checks pass. You ship it. Feels good, right?

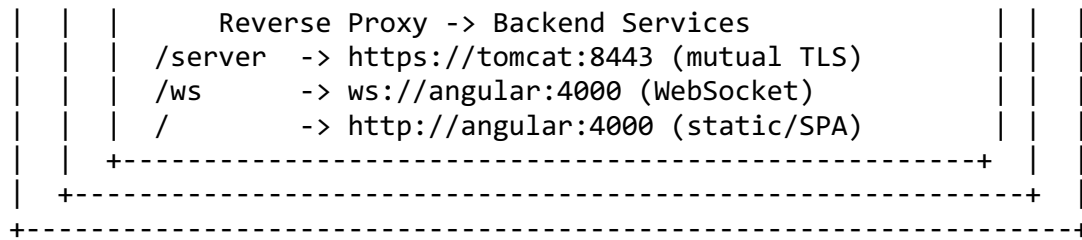
Three months later, a CVE drops for a memory corruption bug in `mod_proxy`. An attacker sends a crafted request, gets a shell inside the container, and... what happens next?

If your container runs as root, they own the host. If it binds to 0.0.0.0, they're scanning your management network. If the filesystem is writable, they're dropping a cryptominer in /usr/bin before you finish your coffee.

You learned this the hard way. This series walks through hardening Apache HTTPD 2.4 in Docker using real production configs – the same ones running in environments where “oops” isn’t an option. You’ll get a Dockerfile, docker-compose.yml, and hardened httpd.conf – plus the “why” behind every decision so you can defend it at 2 AM during an incident review.

The Architecture: Blast Radius Reduction First





Five decisions that do the heavy lifting before application code runs:

Decision	Security Rationale
Bind to specific internal IP (not 0.0.0.0)	Eliminates exposure on unintended interfaces
Run as apache:apache (UID 48)	Drops root after bind; escapes yield unprivileged shell
Read-only root FS + tmpfs for /run, /tmp	Prevents runtime binary drops or config tampering
Separate networks: frontend / backend	WAF sees traffic, backend doesn't
Volume-mounted logs to host	Persistent audit trail survives container replacement

Notice what's *not* here: no WAF rules, no cipher suites, no CSP headers. Those come later. This layer is about **reducing blast radius** so when something gets through, damage is contained.

The Dockerfile: Least Privilege Starts at Build Time

```
FROM docker.io/library/httpd:2.4.62-alpine3.20
```

```
RUN addgroup -g 48 -S apache 2>/dev/null || true && \
    adduser -u 48 -S -G apache -s /sbin/nologin -h /var/www apache
2>/dev/null || true
```

```
COPY conf/httpd.conf /usr/local/apache2/conf/httpd.conf
COPY conf.d/ssl.conf /usr/local/apache2/conf/extra/httpd-ssl.conf
COPY conf.d/mod_security.conf /usr/local/apache2/conf/extra/mod_security.conf
COPY conf.d/mod_qos.config /usr/local/apache2/conf/extra/mod_qos.conf
COPY conf.modules.d/ /usr/local/apache2/conf/modules.d/
COPY modsecurity.d/ /usr/local/apache2/conf/modsecurity.d/
```

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD wget -q --spider --no-check-certificate
https://localhost/server/api/core/health || exit 1
```

USER apache
EXPOSE 80 443

Seven lines that matter:

1. **Pinned Alpine base** – ~5 MB, no glibc, no systemd. Official image creates apache at UID 48. Pinning prevents surprise upgrades breaking hardening.
2. **Idempotent user creation** – `|| true` lets rebuilds succeed. `-s /sbin/nologin` = no interactive login. Home = `/var/www` so no `/home/apache` to abuse.
3. **COPY before USER apache** – Files copied as root are owned by root. apache can read, not modify. First immutability guarantee.
4. **Certificates commented out** – Never bake certs. They rotate, they're secrets, they differ per env. Mount at runtime via compose (read-only).
5. **VOLUME commented, tmpfs in compose** – Dockerfile declares intent; compose enforces it. tmpfs gives writable dirs without making root FS writable.
6. **HEALTHCHECK hits real app endpoint** – If backend is down, container shows unhealthy. Orchestrator restarts or drains it.
7. **USER apache** – Non-root from startup. With `read_only: true` in compose, even a root exploit in a module can't write to filesystem.

What This Means for Your Strategy

Here's the thing: most teams harden the *application* and forget the *web server* sitting in front of it. But Apache is your first line of defense – every request hits it before your code. If Apache is running as root with a writable filesystem and open ports, your app's security doesn't matter. The attacker is already inside.

This architecture shifts the conversation from “how do we prevent every attack?” to “how do we contain the blast when something gets through?” That's a strategy that scales.

What's Coming Next

Part 2 cracks open `httpd.conf`: the three-tier directory model that stops directory listings, SSI injection, and verb tampering. Plus the `FilesMatch` rule catching the `.env` file your developer *definitely* didn't commit to the web root.

Spoiler: it's 30+ extensions and it's saved my bacon more than once.

[Read Part 2 ->](#)

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Non-root container USER (UID 48)	Domain 3: Security Architecture	3.2: Implement secure system design	Least privilege execution reduces blast radius of container escape
Read-only root filesystem + tmpfs	Domain 3: Security Architecture	3.2: Implement secure system design	Immutable infrastructure prevents runtime tampering
Bind to specific internal IP (not 0.0.0.0)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Network segmentation reduces attack surface
Separate Docker networks (frontend/backend)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Microsegmentation limits lateral movement
Volume-mounted persistent logs	Domain 4: Security Operations	4.1: Apply security monitoring concepts	Audit trail survives container lifecycle
Pinned base image with digest	Domain 3: Security Architecture	3.3: Implement secure protocols/services	Supply chain integrity, reproducible builds
HEALTHCHECK against application endpoint	Domain 4: Security Operations	4.5: Implement automation/orchestration	Self-healing infrastructure, automated failover

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Multi-stage build (COPY before USER)	Domain 3: Security Architecture	3.2: Implement secure system design	Build-time vs runtime privilege separation

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~65% | Domain 4 (Operations) – ~35%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives